Week 3 – Architecture Document

Team P3
Cross-Platform Budgeting Application Project
Karunakar Reddy Kaitha, Jordan Lee and John Lim
Computer Science and Information Systems Department, Regis University
MSSE 692: Software Engineering Practicum I
Morgan Worrell
February 1, 2026

I. Executive Summary

The Cross-Platform Budgeting Application adopts a hybrid architectural approach consisting of a modular monolith core supported by selective external services. This architecture balances rapid development, simplicity, and maintainability while remaining flexible enough to support future growth. The system is designed as a client–server architecture with web and mobile clients interacting with a centralized backend API responsible for authentication, business logic, and data persistence.

A modular monolith was deliberately chosen over a full microservices or serverless architecture due to the project's primary constraints: a small development team, an eight-week delivery timeline, and limited infrastructure resources. While microservices offer strong independent scalability and deployment flexibility, they introduce significant operational and cognitive overhead that is unnecessary for the current scope. The selected approach provides clear separation of concerns within the codebase while avoiding the complexity of distributed systems.

The architecture is explicitly driven by the top three quality attribute priorities identified in the requirements document: Performance, Usability, and Security.

- Performance is addressed through a centralized REST API, efficient data access via indexed relational database tables, and optional in-memory caching. This supports the requirement that frequent user actions—such as login, transaction entry, and dashboard updates—complete within ten seconds.
- Usability is supported through thin clients, responsive front-end design, and simplified workflows that minimize user friction. Centralized logic ensures consistent behavior across web and mobile platforms, aligning with cross-device usage scenarios.
- Security is achieved by integrating an external authentication provider, enforcing token-based authorization, and using encrypted HTTPS communication. This reduces the need for custom identity management while ensuring protected financial data remains inaccessible to unauthorized users.

Additional quality attributes, including reliability, are supported through stateless backend services, persistent storage, and graceful error handling to prevent data loss during failures.

The primary trade-offs accepted with this architecture include reduced independent scalability of individual modules and tighter coupling compared to microservice architecture. These trade-offs are considered acceptable given the project's academic context and short timeline. Importantly, the modular design allows for incremental refactoring into microservices in the future if system scale or requirements evolve.

Overall, this architecture provides a strong, realistic foundation for implementation during Weeks 4–8, maintaining clear traceability to documented requirements while remaining achievable within project constraints.

II. Architectural Drivers Analysis

The architecture of the Cross-Platform Budgeting Application is primarily driven by the top three quality attributes identified in the requirements document: Performance, Usability, and Security. These quality attributes were selected as the dominant architectural drivers based on the frequency of related scenarios, their priority in requirements prioritization, and their direct impact on user adoption and trust. Reliability, while still important, is treated as a supporting quality attribute rather than a primary architectural driver.

Driver 1: Performance

Relevant Requirements Scenarios:

- Performance Scenario A (Section 5.1.1): App launch completes within 10 seconds
- Performance Scenario B (Section 5.1.2): Transaction submission and budget recalculation complete within 10 seconds

Architectural Impact:

Performance requirements strongly influenced the decision to adopt a centralized backend API and a modular monolith architecture. Centralized business logic and data access reduce network overhead and simplify optimization strategies such as database indexing and optional in-memory caching. This approach supports predictable and

consistent response times for frequent user actions such as logging expenses and viewing dashboards.

Business Impact if Not Addressed:

If performance requirements are not met, users may experience delays during core workflows, leading to frustration and abandonment. Since the application targets users who expect quick, low-friction interactions, poor performance would directly undermine adoption and continued use.

Driver 2: Usability

Relevant Requirements Scenarios:

- Usability Scenario A (Section 5.2.1): Expense entry completed in under 10 seconds
- Usability Scenario B (Section 5.2.2): New users complete initial setup without assistance in under five minutes

Architectural Impact:

Usability considerations drove the choice of thin clients and a unified backend that enforces consistent workflows across web and mobile platforms. By centralizing business rules and validation logic, the architecture ensures uniform behavior regardless of client type. This supports simple, intuitive user interactions and minimizes user-facing complexity.

Business Impact if Not Addressed:

Poor usability would increase cognitive load and setup friction, particularly for first-time users. Given the competitive nature of consumer budgeting applications, usability failures would likely result in early user drop-off and reduced perceived value of the product.

Driver 3: Security

Relevant Requirements Scenario:

- Security Scenario (Section 5.4.1): Unauthorized access attempts are denied and no protected data is disclosed

Architectural Impact:

Security requirements directly influenced the integration of an external authentication provider, token-based authorization, and encrypted HTTPS communication. Offloading identity management reduces implementation risk while ensuring secure access boundaries around sensitive financial data. These mechanisms are foundational to the system's trust model and data protection strategy.

Business Impact if Not Addressed:

Failure to adequately secure user data would result in loss of user trust and could permanently damage the application's credibility. Given the sensitivity of personal financial information, even a single security breach could render the application unusable from a business perspective.

Supporting Quality Attribute: Reliability

Relevant Requirements Scenarios:

- Reliability Scenario A (Section 5.3.1): Graceful handling of API failures
- Reliability Scenario B (Section 5.3.2): No data loss during system interruptions

Reliability is addressed through stateless backend services, persistent storage, and graceful error handling. While not a primary architectural driver, it complements the primary drivers by ensuring system stability and data integrity.

Summary of Architectural Drivers

The architectural decisions for this project are most heavily influenced by Performance, Usability, and Security, as these quality attributes have the greatest impact on user experience, trust, and adoption. By explicitly addressing these drivers, the architecture ensures alignment with the highest-priority requirements while remaining feasible within the project's constraints

III. Architecture Overview

The Cross-Platform Budgeting Application is designed using a client–server architectural style with a layered, modular monolith backend. This approach was selected to balance simplicity, maintainability, and performance while remaining achievable within the project's eight-week timeline and small team constraints. The architecture supports

both web and mobile clients while centralizing business logic and data management in a single backend system.

Architectural Style and Pattern

At a high level, the system follows a layered client–server architecture. Presentation concerns are handled by separate web and mobile clients, while application logic, security, and data access are centralized in a backend REST API. Internally, the backend is structured as a modular monolith, with clear separation between authentication, business logic, and persistence layers. This structure enables clean boundaries within the codebase without introducing the complexity of distributed microservices.

Key Structural Elements and Responsibilities

The major architectural components and their responsibilities are as follows:

- Web Client: Provides a browser-based user interface for viewing budgets, entering transactions, and reviewing spending trends.
- Mobile Client: Provides a mobile-optimized interface supporting quick, on-the-go expense entry and budget review.
- Backend API: Serves as the central application layer, responsible for request handling, authentication enforcement, business logic execution, and coordination of data access.
- Relational Database: Stores persistent data including user accounts, budget categories, and transaction records, ensuring data integrity through ACID-compliant operations.
- External Authentication Provider: Manages user identity and authentication, reducing security risk and eliminating the need for custom credential storage.

Integration and Communication Approach

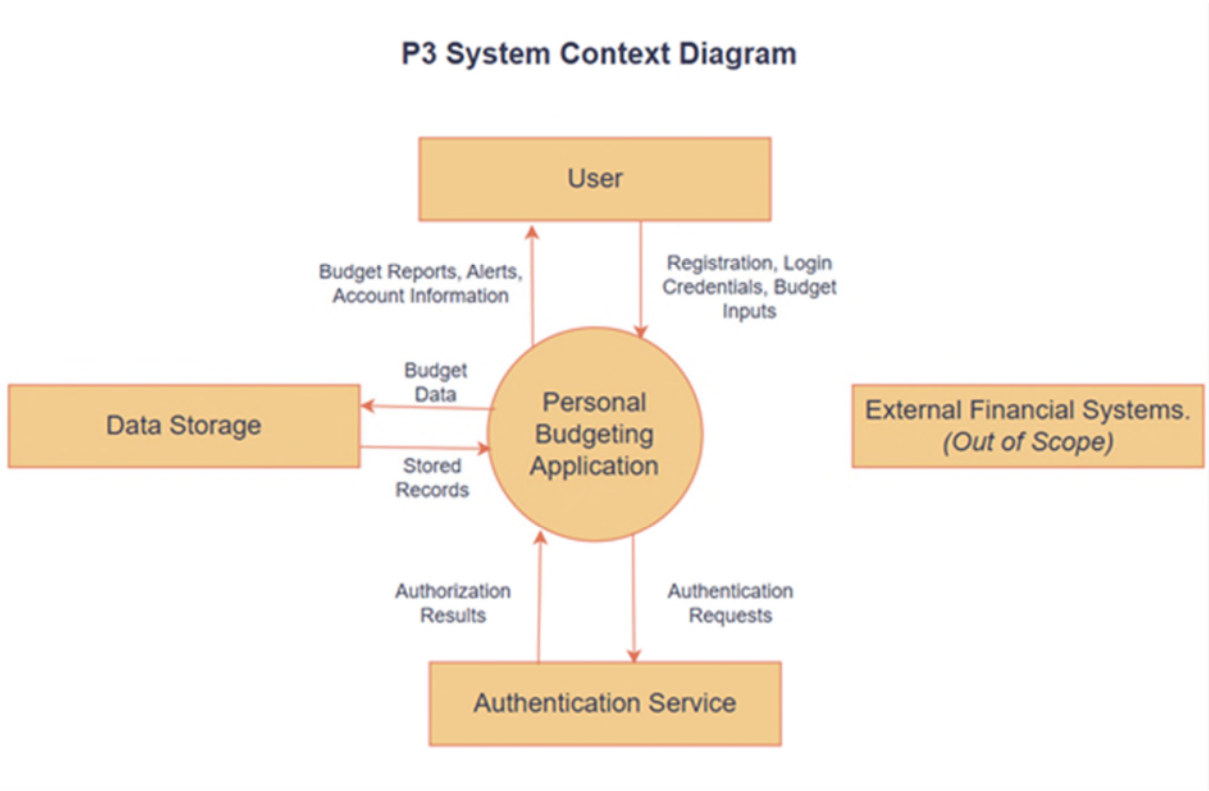
System components communicate using synchronous HTTPS-based REST APIs. Both web and mobile clients interact directly with the backend API for all authenticated operations. Token-based authorization is used to secure requests, and all communication is encrypted to protect sensitive financial data. The backend integrates with the external authentication provider to validate user identity and enforce access control boundaries.

Deployment Architecture

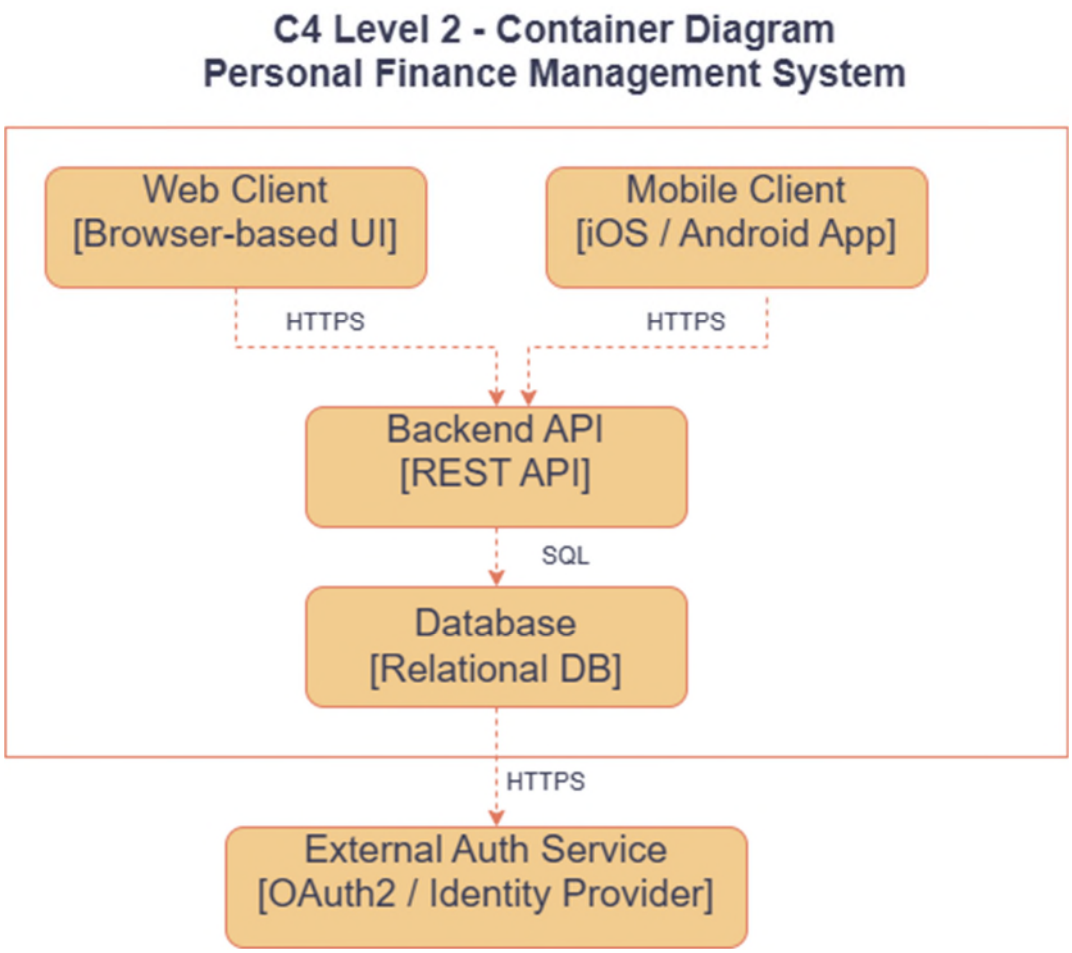
The system is intended to be deployed in a cloud-based environment using containerized services. The backend API and database are deployed as Docker containers, supporting portability and rapid setup within free-tier infrastructure constraints. Clients access the backend over HTTPS, and the deployment model supports straightforward scaling and future evolution without requiring significant architectural changes.

IV. Architecture Views

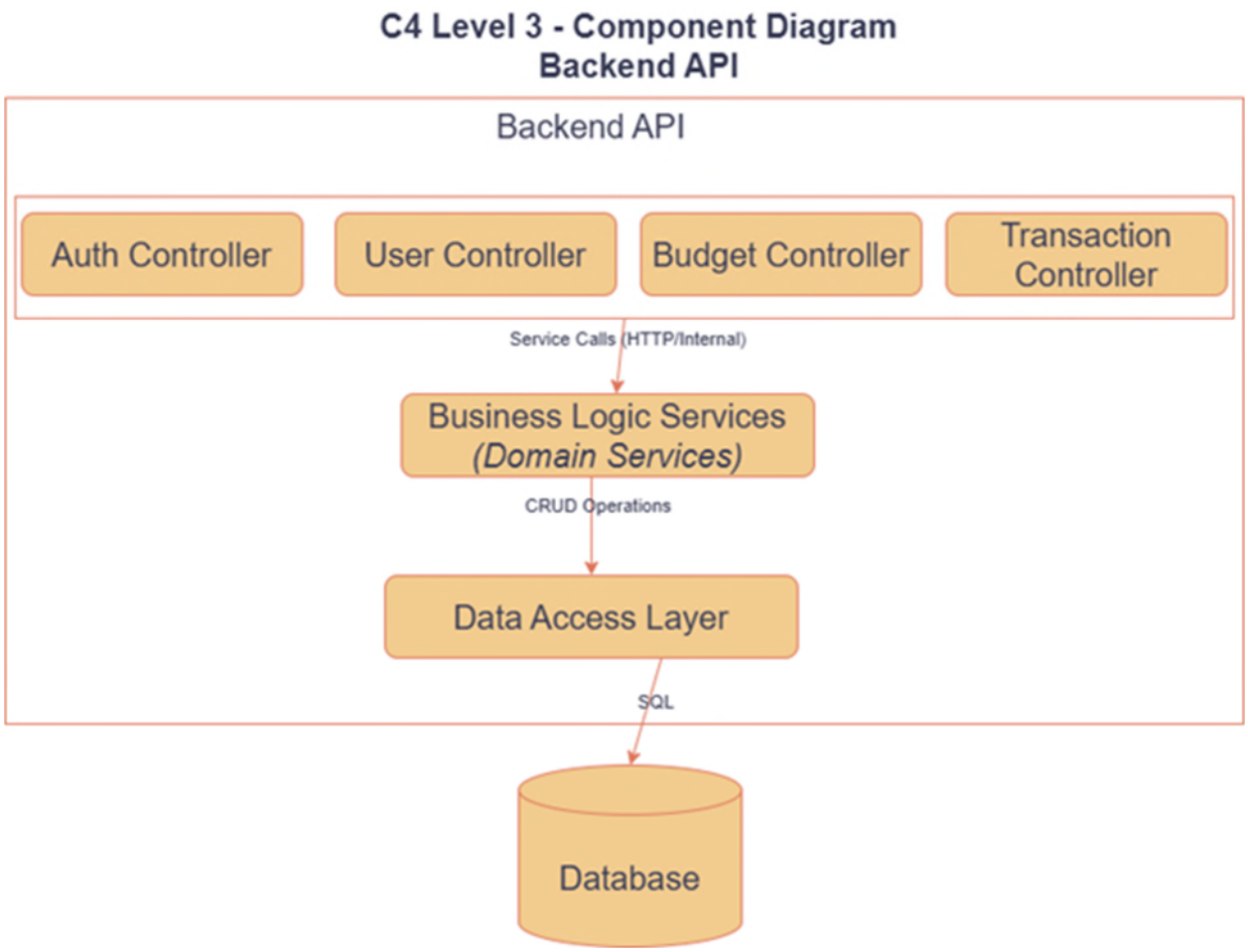
Level 1: System Context Diagram



Level 2: Container Diagram



Level 3: Component Diagram



V. Quality Attribute Achievement Strategy

This section explains how the proposed architecture satisfies the four primary quality attribute scenarios defined in the Requirements Document, using explicit architectural tactics, supporting architecture elements, and a clear stimulus–response mapping.

QA Scenario QA-01: Performance (≤ 10-Second Response Time)

Referenced Requirements Scenarios:

- Performance Scenario A (Section 5.1.1): Initial application screen renders within 10 seconds
- Performance Scenario B (Section 5.1.2): Transaction submission and budget recalculation complete within 10 seconds

Architectural Tactics Applied:

- Centralized backend processing to minimize network hops

- Indexed relational database tables for frequent queries
- Optional in-memory caching for commonly accessed data
- Thin client design to reduce client-side computation

Supporting Architecture Elements:

- Backend REST API
- Relational database
- Database indexing on transaction and category tables
- Optional in-memory cache layer

How This Achieves the Scenario:

When a user launches the application or submits a transaction (stimulus), the client sends a synchronous request to the centralized backend API. Optimized database queries and optional cached data allow the backend to process requests efficiently and return results within the required 10-second response window. This ensures that core user actions such as login, expense entry, and dashboard updates consistently meet performance requirements.

QA Scenario QA-02: Usability (Low-Friction User Interaction)

Referenced Requirements Scenarios:

- Usability Scenario A (Section 5.2.1): Expense entry completed in under 10 seconds
- Usability Scenario B (Section 5.2.2): Initial user setup completed in under five minutes without assistance

Architectural Tactics Applied:

- Thin client architecture with centralized business logic
- Consistent workflows enforced by backend validation
- Shared API for web and mobile clients to ensure parity
- Simplified request/response interactions

Supporting Architecture Elements:

- Web client
- Mobile client
- Backend REST API

- Shared validation and business logic modules

How This Achieves the Scenario:

When users enter expenses or complete initial setup (stimulus), the architecture minimizes required steps by centralizing rules and workflows in the backend. Clients present simple, guided interfaces while the backend ensures consistent behavior across platforms. This reduces cognitive load, shortens interaction time, and enables users to complete common tasks quickly and intuitively.

QA Scenario QA-03: Reliability (No Data Loss and Graceful Failure)

Referenced Requirements Scenarios:

- Reliability Scenario A (Section 5.3.1): Graceful handling of API request failure
- Reliability Scenario B (Section 5.3.2): No data loss during unexpected system interruption

Architectural Tactics Applied:

- Stateless backend service design
- Persistent storage for all user and transaction data
- Graceful error handling with retry prompts
- ACID-compliant database transactions

Supporting Architecture Elements:

- Stateless backend REST API
- Relational database
- Error handling and retry logic in client applications

How This Achieves the Scenario:

If an API request fails or a system interruption occurs (stimulus), the stateless backend ensures no session data is lost, and previously committed data remains intact in persistent storage. Clients receive clear error messages and retry options, preventing crashes and ensuring that user data is preserved under failure conditions.

QA Scenario QA-04: Security (Protection of Sensitive Financial Data)

Referenced Requirements Scenario:

- Security Scenario (Section 5.4.1): Unauthorized access attempts are denied and no protected data is disclosed

Architectural Tactics Applied:

- Externalized authentication and identity management
- Token-based authorization
- Encrypted communication via HTTPS
- Strict access control enforcement at API boundaries

Supporting Architecture Elements:

- External authentication provider
- Backend API authorization middleware
- Token mechanism
- HTTPS communication channel

How This Achieves the Scenario:

When an unauthorized access attempt occurs (stimulus), authentication tokens are validated by the backend API before any protected resource is accessed. Requests without valid credentials are rejected, and all communication is encrypted in transit. This prevents disclosure of sensitive financial data and enforces strong trust boundaries throughout the system.

Summary of Quality Attribute Achievement Strategy

By using targeted architectural tactics and clearly defined supporting elements, the system architecture explicitly satisfies the Performance, Usability, Reliability, and Security quality attribute scenarios defined in the Requirements Document. Each quality attribute is directly traceable to architectural decisions, ensuring that the system behavior aligns with the project's highest priority requirements.

VI. Technology Selection Rationale

This section documents the key technology choices for the Cross-Platform Budgeting Application and explains how each choice supports the system requirements, quality attributes, and project constraints, including a small team and an eight-week development timeline.

Frontend Frameworks

Technologies Selected:

- React (Web Client)
- React Native (Mobile Client)

Rationale:

React and React Native were selected for the client applications because:

- Team Experience and Delivery Risk Reduction: The development team has prior experience with React-based frameworks, reducing onboarding time and minimizing implementation risk within the constrained project timeline.
- Usability Support: React's component-based architecture enables rapid iteration of user interfaces, directly supporting usability requirements such as fast expense entry and intuitive navigation.
- Cross-Platform Consistency: React Native allows reuse of design patterns and business logic concepts across web and mobile clients, supporting consistent behavior and user experience for cross-device users.
- Maintainability: Modular UI components improve maintainability and allow future feature expansion without large refactoring efforts.

Backend Language and Framework

Technologies Selected:

- Node.js
- Express.js

Rationale:

Node.js with Express was selected for the backend API because:

- Performance and Responsiveness: Node.js's event-driven, non-blocking I/O model supports handling multiple concurrent requests efficiently, aligning with performance scenarios requiring responses within ten seconds.
- Stateless Service Design: Express supports lightweight, stateless REST services, which simplifies scaling and improves reliability by avoiding server-side session dependencies.

- Rapid API Development: Express provides a minimal yet flexible framework for implementing REST endpoints quickly, supporting the project's aggressive delivery timeline.

Database Technology

Technology Selected:

- PostgreSQL

Rationale:

A relational database was chosen due to the structured and transactional nature of budgeting data. PostgreSQL was selected specifically because:

- Data Integrity and Reliability: ACID compliance ensures that financial records such as transactions and budgets are stored reliably with no data loss during failures.
- Structured Data Model: The application relies on well-defined relationships between users, categories, and transactions, which are naturally modeled using relational schemas.
- Performance Optimization: Indexing and query optimization support frequent read and write operations required by dashboard views and budget calculations.
- Maturity and Stability: PostgreSQL is a proven, production-ready database suitable for persistent financial data.

Infrastructure and Deployment Platform

Technologies Selected:

- Docker

Rationale:

Docker was selected as the deployment mechanism because:

- Environment Consistency: Containers ensure consistent runtime environments across development and deployment, reducing configuration-related defects.
- Rapid Setup and Portability: Containerization supports quick setup and teardown, which is critical in an academic, time-constrained project.
- Alignment with Free-Tier Infrastructure: Docker enables flexible deployment on free-tier or low-cost cloud environments without vendor lock-in.

- Future Scalability: Containerized services provide a foundation for future orchestration or scaling if project scope expands.

Integration Technologies

Technologies Selected:

- RESTful APIs over HTTPS
- JWT-based authentication

Rationale:

REST over HTTPS was selected as the primary integration mechanism because:

- Simplicity and Interoperability: REST APIs are widely supported and easy to consume from both web and mobile clients.
- Clear Separation of Concerns: RESTful interfaces enforce clean boundaries between clients and backend services.
- Security: HTTPS encryption and JWT-based authorization protect sensitive financial data during transmission and enforce access control.
- Scalability: Stateless REST interactions support horizontal scaling and align with the system's modular monolith design.

Summary of Technology Selection Rationale

The selected technologies collectively support the system's performance, usability, security, and reliability requirements while remaining realistic given the project's time, scope, and team constraints. Each technology choice prioritizes simplicity, maintainability, and low delivery risk, providing a strong foundation for implementation and future evolution.

VII. Architectural Constraints and Assumptions

This section identifies the primary constraints and assumptions that shape the system architecture, along with key architectural risks and the strategies used to mitigate them.

Architectural Constraints

The following constraints limit the range of feasible architectural options:

- Project Timeline: The system must be designed and implemented within an eight-week practicum. This constraint limits the feasibility of complex or highly distributed architecture that require significant setup and operational overhead.
- Team Size and Skills: The project is developed by a small team with existing experience in JavaScript-based technologies. Architectural choices must align with these skills to reduce onboarding time and delivery risk.
- Infrastructure Budget: The application must run on free-tier or low-cost infrastructure. This constraint restricts the use of managed enterprise services and favors lightweight, container-based deployment models.
- Project Scope: The application targets a minimal viable feature set, excluding bank integrations and large-scale production deployment. The architecture must therefore remain simple while supporting core budgeting functionality.

These constraints directly influenced the decision to adopt a modular monolith architecture with centralized services.

Architectural Assumptions

The architecture is based on the following assumptions:

- Infrastructure Availability: A cloud-based runtime environment capable of hosting containerized services is assumed to be available throughout development and testing.
- Third-Party Service Reliability: External authentication services are assumed to be reliable and available during normal operation.
- User Behavior: Users are assumed to manually enter income and expense data and to perform short, frequent interactions rather than long sessions.
- System Load: The system is expected to support a limited number of concurrent users consistent with an academic prototype, rather than production-scale traffic.

If these assumptions change, architectural adjustments may be required.

Architectural Risks and Mitigation Strategies

The following architectural risks have been identified, along with their mitigation strategies:

- Performance Degradation: Increased response times during frequent operations such as expense entry or dashboard loading, Mitigation Strategy: Use indexed database queries, optional in-memory caching, and centralized backend processing
- Dependency on External Authentication: Outages or latency in the authentication provider could prevent user access, Mitigation Strategy: Use token-based authentication with reasonable expiration and clear error handling for authentication failures
- Data Loss or Corruption: Unexpected failures could result in loss of financial data, Mitigation Strategy: Use ACID-compliant relational database, persistent storage, and transactional writes
- Deployment Misconfiguration: Inconsistent environments could cause runtime errors, Mitigation Strategy: Use containerized deployment with Docker to ensure environment consistency
- Scope Expansion: Additional features could introduce architectural complexity beyond project constraints, Mitigation Strategy: Maintain strict scope control and modular code organization to allow incremental extension.

Summary of Architectural Constraints and Assumptions

The documented constraints and assumptions establish clear boundaries for architectural decision-making, while identified risks and mitigation strategies reduce the likelihood of architectural failure. Together, these elements ensure that the architecture remains realistic, resilient, and aligned with project goals and constraints.

VIII. Team Architecture Review Summary

Date of Architecture Review – February 1, 2026

Attendees – Jordan Lee, John Lim, Karunakar Reddy Kaitha

Feedback Received

During the review, John Lim provided feedback focused on improving the architectural documentation to better align with the assignment instructions. The feedback emphasized that while the existing architecture was technically sound, several sections would benefit from clearer structure, stronger traceability to the requirements document,

and more explicit justification of architectural decisions.

Key feedback points included:

- The need to explicitly identify and prioritize architectural drivers based on the top-quality attribute scenarios from the requirements document.
- The need for clearer, scenario-based explanations of how architectural tactics satisfy quality attribute requirements.
- Additional clarity in the Architecture Overview and Technology Selection Rationale sections.
- More explicit documentation of architectural constraints, assumptions, risks, and corresponding mitigation strategies.

Changes Made Based on Feedback

Based on this feedback, the following improvements were implemented in the architecture document:

- **Executive Summary Enhancement**

The Executive Summary was revised to explicitly identify the architectural approach, map the top three quality attribute priorities to architectural decisions, and clearly document accepted trade-offs.

- **Architectural Drivers Analysis Revision**

The Architectural Drivers Analysis section was rewritten to explicitly identify Performance, Usability, and Security as the top three architectural drivers. Each driver now references specific quality attribute scenarios from the requirements document and explains the business impact if the driver is not addressed architecturally.

- **Architecture Overview Clarification**

The Architecture Overview section was expanded to clearly describe the architectural style, major system components and responsibilities, integration mechanisms, and deployment approach.

- **Quality Attribute Achievement Strategy Revision**

The Quality Attribute Achievement Strategy section was restructured to follow a consistent scenario-based format. For each quality attribute, the section now documents

the applicable requirement scenarios, architectural tactics, supporting architecture elements, and a clear explanation of how the architecture satisfies each scenario.

- **Technology Selection Rationale Strengthening**

The Technology Selection Rationale section was revised to provide explicit, requirement-driven justifications for all major technology choices, including frontend frameworks, backend technologies, database selection, deployment approach, and integration mechanisms.

- **Constraints, Assumptions, and Risks Expansion**

The Constraints, Assumptions, and Risks section was expanded to clearly distinguish between constraints and assumptions, identify multiple architectural risks, and document concrete mitigation strategies for each identified risk. (edited)

Team Approval and Consensus

Following discussion of the proposed changes, all team members reached consensus and formally approved the revised architecture during the review session held on February 1, 2026. The team agreed that the updated architecture document accurately reflects project requirements, constraints, and quality attribute priorities, and approved it as the authoritative architectural baseline for subsequent implementation activities.